

● CLICK-A-THON 2026 · INMOBI TRACK

Sherlook

From alert to answer.

An automated root-cause analyst for an ad marketplace. It finds what broke, states what it checked and ruled out, prices the damage, and says what to do — with deterministic code doing the analysis and the LLM only narrating.

The Boring Team — Sam Melvin M · Loges S · Samarth Dhawan · Mohansundar K

THE PROBLEM

When a metric drops, someone spends the morning slicing dashboards.

- An ad marketplace has **thousands of segments** — app, device, OS, geo, advertiser, ad format — any combination of which can be the actual cause of a metric move.
- A manual investigation means opening dashboards, guessing which cut to slice by, and hoping the real cause isn't hiding behind three correlated red herrings.
- Worse: a wrong diagnosis is expensive. **A single fabricated or misattributed number costs more than a missed anomaly** — it sends the wrong team chasing the wrong fix.
- What's needed isn't a smarter dashboard. It's a system that **investigates like an analyst would** — and can prove its own work.

WHAT SHERLOOK DOES

Four questions, answered automatically, every time.

What broke?

Every metric, every day,
swept against a same-
weekday trailing baseline.

Where?

The exact segment(s)
responsible — with every
other suspect explicitly
checked and cleared.

How much?

Dollars per day, priced from
the same evidence that
named the cause.

What now?

An owner and a funnel stage
to look at — a diagnosis with
no next step is half an answer.

THE PIPELINE

Six stages. Every one of them SQL.



- **Detect** — same-weekday baseline, median + MAD (never mean/stddev — an incident in the trailing window inflates the standard deviation and hides itself).
- **Decompose** — which part of the funnel moved: requests → fill → render → CTR → price.
- **Localize** — which segment values, across 8 dimensions and their pairs, carry the move.
- **Residualize** — the differentiator, next slide.
- **Confirm & classify** — still significant once the cause is excluded? Technical break, mix shift, or demand — with an owner and dollars/day.

THE DIFFERENTIATOR

When one segment breaks, everything containing it looks broken too.

Android 15's fill rate collapsed. Because Android 15 sits inside Europe, and inside a popular device model, a naive drill-down blames Europe, the device, and a dozen other slices that never actually moved on their own.

Residualization removes the cause's contribution and re-measures everything else. Segments that only looked broken because they contain the cause are **explicitly cleared**, with the residual as proof — not silently dropped.

840

SLICES CLEARED, ONE INCIDENT

78% → 43%

ANDROID 15 FILL RATE, THE REAL CAUSE

9.6%

OF TRAFFIC — THE TRUE BLAST RADIUS

TRUSTWORTHINESS IS A GATE, NOT AN ASPIRATION

The LLM cannot write SQL. There is no tool that lets it.

- The MCP server exposes **13 typed tools** — `describe_data`, `get_metric`, `compare_periods`, `rank_segments`, `find_incidents`, `investigate`, `explain_revenue`, `get_evidence`, `export_trace...` — and not one accepts a raw query.
- Enforced by a **gate**, not a promise: `bun run sanity` fails the build if any tool name matches `/sql|query|exec|raw/`.
- A **grounding check** re-reads the finished diagnosis and maps every numeral back to a recorded evidence row, at the precision printed. A number that can't be traced is a failure.
- The model's entire job: read a JSON result, write a paragraph. That's the whole failure surface — small enough to actually test.

WHY CLICKHOUSE

Rows read and rows returned are bounded by cardinality, not by event volume.

58x

FEWER ROWS READ, ROLLUP VS. RAW

45x

FASTER SERVER TIME

5 / 6

INVESTIGATION STAGES ROLLUP-SERVED

Two materialized views, maintained on insert: `ad_events` → `mv_rollup_segment_hourly` (3.09M rows) → `mv_rollup_segment_daily` (148.8K rows), the daily cascading from the hourly rather than a second independent aggregation — so the two grains cannot drift apart. `bun run parity` runs every investigation twice, rollup vs. raw-forced, and asserts every recorded number is identical.

All three. None of them decorative.

ClickStack

Every stage, every tool call, every SQL statement is a span — with the statement text and row count attached. A slow answer traces down to the exact query that made it slow.

Langfuse

Real production cost/token attribution per model. The dashboard's own "Recent Prompts" view reads Langfuse's API server-side to show real prompts, real cost, and what was actually checked for each one.

LibreChat

The chat surface, talking to the MCP server over streamable HTTP. User identity flows through as headers, so a scheduled watch belongs to a person and follow-ups carry the same evidence as the first answer.

What actual Langfuse traces show about cost and latency.

- Backend `investigate()` calls, sampled across real prompts: **0.5s – 28s**, with the one outlier fully explained (three concurrent investigations sharing a connection pool, not a slow query).
- A full, clean single-pass investigation costs **~14.4 seconds** end to end — matching the "a few seconds per investigation" design target.
- What actually drives a slow "hard" answer isn't backend latency — it's how many drill-down rounds the agent chooses to take. Fixed by capping how many candidate windows a sweep hands the agent at once, and telling it to investigate the top few first.
- LLM cost per generation: fractions of a cent. The expensive part was never the model.

VERIFICATION

Not "it worked when we tried it" — a gate suite that runs every time.

GATE	RESULT
mcp:eval	16/16 cases, 60/60 gated accuracy
criteria	criteria.failed = 0 / 4
ch:verify-rollup	283 probes compared, 265 served from rollup
parity	all 5 scenarios rollup-served, every number matched raw
synth:verify	0 gated failures on data the engine has never seen
mcp:handshake	13 tools, no SQL escape hatch
secrets	no credential in any tracked file

One command. No arguments. Nothing hand-written.

```
$ bun run diagnose  
  sweep everything → rank & cut → group → investigate top N → report
```

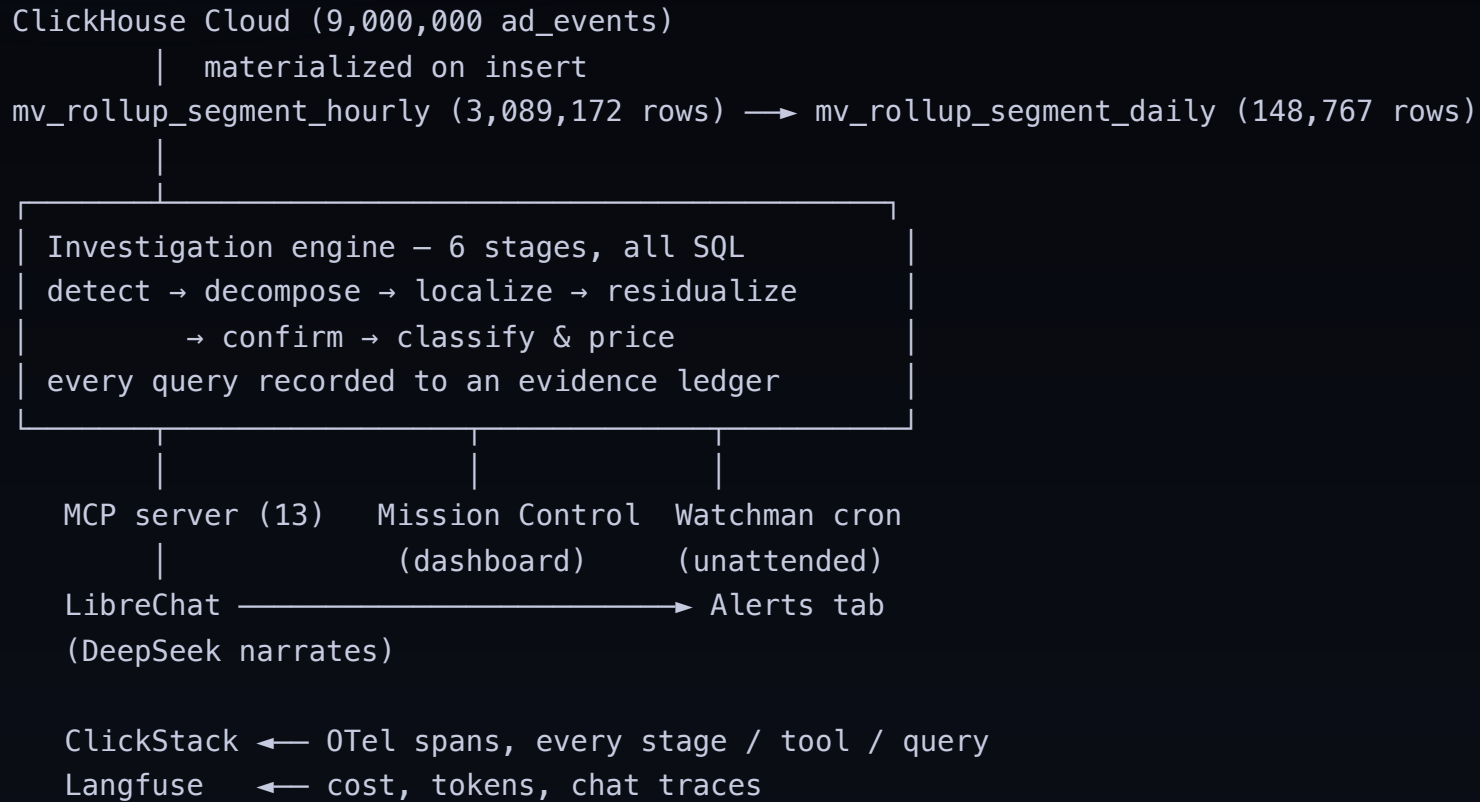
- Tested against **synthetic data with planted deviations the engine has never seen** (`synth:build` / `synth:verify`) — so tuning to the training incidents was never the goal.
- **Rank and cut:** every firing window is scored and listed, not just the escalated ones — a digest that silently drops what it saw is how a real incident gets missed.
- Output is `report.md` (diagnosis), `report.json` (every number, with its evidence id), and the trace file — all three, by construction, not written after the fact.

Trustworthiness beats sophistication — including about our own gaps.

- **Known** Action-recommendation figures aren't yet in the evidence ledger — caught by our own grounding gate, not by a judge.
- **Known** Narration isn't fully deterministic even at temperature 0 — occasionally flags a correct rounding as ungrounded rather than passing something wrong.
- **Known** LibreChat's agent trace and our backend trace don't yet share an OpenTelemetry traceId in production — root-caused to one unset env var on LibreChat's side, not a design flaw; our own extraction code already handles it correctly.
- Every one of these was found by **our own gates**, not discovered by a judge — that's the point of building the gates.

ARCHITECTURE, AT A GLANCE

ClickHouse does the analysis. Everything else observes or narrates it.



CLOSE

Deterministic code finds the answer. The model reads it out.

That's the whole idea. Every number on screen came out of ClickHouse on request — nothing here is a mock.

Sherlock — The Boring Team — Click-a-thon 2026, InMobi track